Green AI Model Benchmarking Framework

Complete Project Report, Concept Notes, Code Understanding, Results, and Research Limitations

Prepared from the work completed so far

Report date: 25 September 2026

Project focus: Compare machine-learning models not only by predictive performance, but also by computational cost and resource-related measurements.

1. Executive Summary

This project develops a small but research-oriented benchmarking framework for Green AI. Instead of judging a machine-learning model only by accuracy, the project studies a wider set of questions: How accurate is the model? How long does it take to train? How long does it take to make predictions? How consistent are the measurements across repeated runs? What additional resource measurements, such as memory and energy, can be added later?

The current experiments use the Breast Cancer Wisconsin Diagnostic dataset available through scikit-learn. Three classification models are compared: Logistic Regression, Random Forest, and Support Vector Machine (SVM). The completed work includes environment setup, a basic timing experiment, a baseline machine-learning experiment, result plotting, and a repeated benchmark with averages and standard deviations.

> **Important conclusion:** the current project measures accuracy and execution time reliably enough for an initial experiment, but it does not yet directly measure electrical energy consumption. Time and CPU usage are related to resource consumption, but they are not the same as energy.

2. Research Question and Project Objectives

Main research question: How does model architecture and complexity affect predictive performance, computational cost, and energy-related measurements in a controlled machine-learning experiment?

Objectives completed so far

• Set up an isolated Python virtual environment.

• Install and verify the main scientific Python libraries.

• Create a reproducible dataset split using random_state=42.

• Train and evaluate Logistic Regression, Random Forest, and SVM.

• Measure training time and prediction time.

• Save results as CSV files for later analysis.

• Create comparison plots for accuracy and training time.

• Repeat the benchmark five times.

• Calculate average values and standard deviations to study variability.

• Identify limitations before adding memory and energy-related measurements.

Objectives planned for the next stage

• Add process-level memory measurements using psutil.

• Document the difference between RSS memory, peak memory, and model-only memory.

• Investigate energy measurement options and their limitations on a CPU-only Windows system.

• Improve experiment configuration and reproducibility.

• Add stronger visualizations and research-style documentation.

• Add tests and a polished README for GitHub.

3. Development Environment and Setup

Component	Current setup
Operating system	Windows
Python	3.12.10

Component	Current setup
Execution environment	Python virtual environment activated in PowerShell
CPU/GPU context	CPU-only workflow
NumPy	2.2.6
Pandas	3.0.2
scikit-learn	1.8.0
Matplotlib	3.10.8
psutil	7.2.2

Why use a virtual environment?

A virtual environment creates an isolated package space for one project. This prevents the project from depending on every package installed globally on the computer. It also improves reproducibility because the installed versions can be recorded in requirements.txt.

requirements.txt

The command used was:

```
python -m pip freeze > requirements.txt
```

The command records installed packages and versions in a text file. Another person can use that file as a starting point for recreating the environment with pip install -r requirements.txt. Exact reproducibility can still depend on the operating system, Python version, hardware, and package compatibility.

4. Project Structure

```
green-ai-benchmark/
■■■ README.md
■■■ requirements.txt
■■■ .gitignore
■■■ config.yaml
■■■ data/
■   ■■■ README.md
■■■ src/
■   ■■■ __init__.py
■   ■■■ data_loader.py
■   ■■■ preprocessing.py
■   ■■■ models.py
■   ■■■ trainer.py
■   ■■■ evaluator.py
■   ■■■ resource_monitor.py
■   ■■■ experiment_runner.py
■■■ experiments/
■   ■■■ configs/
■■■ results/
■   ■■■ metrics/
■   ■■■ plots/
■■■ notebooks/
■   ■■■ exploratory_analysis.ipynb
■■■ tests/
■■■ docs/
    ■■■ methodology.md
```

The experiments completed so far were first developed as separate Python scripts. This is a sensible beginner workflow because each script has a clear purpose. Later, the scripts can be refactored into the proposed src/ structure for maintainability and reuse.

5. Core Concepts

5.1 Green AI

Green AI is an approach to artificial intelligence that considers the environmental and computational cost of developing and using models. A model can be accurate but expensive to train, slow to run, memory-intensive, or energy-intensive. Green AI tries to make these trade-offs visible.

The project therefore treats model evaluation as multi-dimensional:

Dimension	Meaning
Predictive performance	How well the model predicts the target labels.
Training time	Time required to fit the model on training data.
Prediction time	Time required to generate predictions for test data.
Memory	Amount of system memory associated with the process or model execution.
Energy	Electrical energy consumed during computation, ideally measured in joules or watt-hours.

5.2 Classification

The dataset is a binary classification problem. The model predicts one of two possible classes. Accuracy is calculated as the number of correct predictions divided by the total number of predictions.

Formula:

Accuracy = Correct predictions / Total predictions

For example, if a model correctly predicts 112 out of 114 test samples, its accuracy is 112/114, approximately 0.9825 or 98.25%.

5.3 Training versus prediction

• **Training:** the model learns patterns from the training data by fitting its internal parameters.

• **Prediction:** the already-trained model uses learned patterns to produce outputs for new data.

• A model may train quickly but predict slowly, or train slowly but predict quickly. Both phases should therefore be measured separately.

5.4 Time is not energy

Execution time measures duration. Energy measures the amount of electrical work consumed. A simplified relationship is:

Energy = Power × Time

If a computer uses 50 watts for 10 seconds, the approximate energy is 500 joules, because one watt equals one joule per second. In a real system, power can change during execution, so accurate energy estimation requires suitable hardware or software monitoring and careful experimental control.

6. Dataset Understanding

The project uses the Breast Cancer Wisconsin Diagnostic dataset provided through scikit-learn.

Property	Value
Number of samples	569
Number of features	30 numeric features
Task	Binary classification
Split	80% training and 20% testing
Split method	Stratified train-test split

Property	Value
Random seed	42

Why use a stratified split?

A stratified split tries to preserve the class distribution in both the training and testing sets. This is important in classification because an unbalanced split could make the test result less representative.

Why use random_state=42?

The random_state value fixes the random process so that the same code generally produces the same train-test split. The number 42 has no special mathematical meaning here; it is simply a chosen fixed seed for reproducibility.

7. Models Used

7.1 Logistic Regression

Logistic Regression is a relatively simple classification algorithm. It learns a relationship between input features and the probability of a class. Despite its name, it is commonly used for classification.

In this project, Logistic Regression is placed inside a StandardScaler pipeline. Scaling is useful because the input features can have different numerical ranges.

7.2 Random Forest

Random Forest is an ensemble model made from multiple decision trees. Each tree learns patterns, and the forest combines their predictions. The experiment uses n_estimators=100, meaning 100 trees are created.

Random Forest can capture non-linear relationships, but it may require more computation than a simple linear model, especially during training and prediction.

7.3 Support Vector Machine

Support Vector Machine, or SVM, attempts to find a decision boundary that separates classes. The SVC implementation used in the project is placed inside a StandardScaler pipeline.

SVM can perform well on datasets with numeric features, but its computational cost can vary with dataset size, kernel choice, and parameter settings.

8. Understanding the Baseline Code

8.1 Loading the dataset

```
from sklearn.datasets import load_breast_cancer

data = load_breast_cancer()
X = data.data
y = data.target
```

X contains the input features. y contains the target labels. In machine learning notation, X is usually called the feature matrix and y is the target vector.

8.2 Train-test split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

The test_size=0.2 setting reserves 20% of the data for evaluation. The model is trained only on X_train and y_train, then evaluated using X_test and y_test.

8.3 Pipeline and scaling

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

model = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(max_iter=5000))
])
```

The pipeline applies StandardScaler before LogisticRegression. This avoids manually transforming the data in separate steps and reduces the risk of accidentally applying inconsistent preprocessing.

8.4 Timing with time.perf_counter

```
start_time = time.perf_counter()
model.fit(X_train, y_train)
training_time = time.perf_counter() - start_time

start_time = time.perf_counter()
predictions = model.predict(X_test)
prediction_time = time.perf_counter() - start_time
```

time.perf_counter() is appropriate for measuring elapsed duration because it is designed for performance timing. The measured time includes the operations inside the timed block, so the code should keep the block focused.

8.5 Accuracy calculation

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, predictions)
```

accuracy_score compares the true labels with the predicted labels. The result is a value between 0 and 1, where 1 means every prediction was correct.

8.6 Storing results in a DataFrame

```
results.append({
    "Model": model_name,
    "Accuracy": accuracy,
    "Training Time (seconds)": training_time,
    "Prediction Time (seconds)": prediction_time
})

results_df = pd.DataFrame(results)
results_df.to_csv(
    "results/metrics/baseline_results.csv",
    index=False
)
```

A list of dictionaries is convenient because every dictionary represents one experiment result. Pandas converts this list into a table that can be analyzed, plotted, and saved as CSV.

9. First Timing Experiment

Before measuring machine-learning models, a basic Python computation was timed. The program performed a loop and calculated a large sum.

Measurement	Observed result
Computation result	49999995000000
Execution time	Approximately 2.1668 seconds
CPU cores	8
CPU usage reading	Approximately 77.4%

This experiment helped demonstrate how to measure elapsed time and inspect CPU information using psutil.

> **Important limitation:** CPU usage was measured after the computation using psutil.cpu_percent(interval=1). Therefore, it is not a precise record of CPU utilization throughout the whole computation. Also, CPU utilization percentage is not the same as energy consumption.

10. Baseline Benchmark Results

Model	Accuracy	Training time (s)	Prediction time (s)
Logistic Regression	0.982456	0.012239	0.000480
Random Forest	0.956140	0.189844	0.029174
SVM	0.982456	0.005545	0.002022

The baseline experiment suggests that Logistic Regression and SVM achieved the same recorded accuracy on the fixed test split, while Random Forest had lower accuracy in this particular run. Random Forest also took longer to train and predict in the recorded baseline.

These results describe this dataset, implementation, parameter configuration, hardware environment, and test split. They should not be generalized into claims that one algorithm is always better or always more efficient.

11. Plotting and Visualization

The plotting script reads the saved CSV file and creates two figures:

• accuracy_comparison.png — compares accuracy across the three models.

• training_time_comparison.png — compares training time across the three models.

```
results_df = pd.read_csv(
    "results/metrics/baseline_results.csv"
)

plt.bar(results_df["Model"], results_df["Accuracy"])
plt.title("Accuracy Comparison")
plt.ylabel("Accuracy")
plt.xticks(rotation=20)
plt.tight_layout()
plt.savefig("results/plots/accuracy_comparison.png")
plt.close()
```

The use of plt.close() is helpful when producing multiple figures in one script. It closes the current figure and prevents later plots from being mixed with the previous figure.

Earlier, plt.show() caused the script to pause at the first graph until the graph window was closed. Saving figures directly with plt.savefig() is more convenient for a GitHub project because the images can be stored and displayed in the repository.

12. Repeated Benchmarking

A single timing result can be affected by background applications, operating-system scheduling, CPU frequency changes, caching, and other temporary conditions. To reduce dependence on one run, the benchmark was repeated five times.

Why create fresh model instances?

```
def create_models():
    return {
        "Logistic Regression": Pipeline([...]),
        "Random Forest": RandomForestClassifier(...),
        "SVM": Pipeline([...])
    }
```

Fresh model instances prevent a previously fitted model from being reused accidentally. Each run should represent a new training operation.

Summary statistics

```
summary_df = results_df.groupby("Model").agg(
    Average_Accuracy=("Accuracy", "mean"),
    Average_Training_Time=("Training Time (seconds)", "mean"),
    Training_Time_Std=("Training Time (seconds)", "std"),
    Average_Prediction_Time=("Prediction Time (seconds)", "mean"),
    Prediction_Time_Std=("Prediction Time (seconds)", "std")
).reset_index()
```

The mean gives the average observed value across the five runs. Standard deviation describes how spread out the measurements are. A larger standard deviation means the measurements varied more between runs.

The standard deviation is not proof of causation or a complete uncertainty analysis. Five runs are useful for an initial benchmark, but more runs and controlled conditions may be needed for stronger research conclusions.

13. Repeated Benchmark Results

Model	Average accuracy	Average training time (s)	Training std	Average prediction time (s)	Prediction std
Logistic Regression	0.982456	0.009831	0.001271	0.000745	0.000220
Random Forest	0.956140	0.189237	0.016837	0.037707	0.016786
SVM	0.982456	0.005318	0.000709	0.002186	0.000173

Interpretation of the repeated results:

• Logistic Regression and SVM produced the same average accuracy on the fixed split.

• SVM had the lowest average training time in this specific experiment.

• Random Forest had the highest average training time and average prediction time among the three models.

• Random Forest prediction time showed relatively high variation, partly because one run had a noticeably larger prediction-time value.

• The results support comparing multiple dimensions instead of looking at accuracy alone.

> The correct wording is that SVM was fastest in this experiment, not that SVM is always the fastest algorithm. Hardware, software versions, dataset size, preprocessing, hyperparameters, and system load can change the result.

14. Understanding Variability and Fair Comparison

Factors that can influence timing

• Background applications and operating-system processes.

• CPU frequency scaling and thermal conditions.

• First-run overhead from imports, memory allocation, and library initialization.

• Number of CPU threads used by a model.

• Garbage collection and memory allocation behavior.

• Dataset size, feature count, model parameters, and implementation details.

How to improve fairness

• Use the same dataset split for every model.

• Use the same hardware and software environment.

• Use fresh model instances for every run.

• Run multiple repetitions.

• Report mean and standard deviation.

• Keep preprocessing inside the model pipeline when appropriate.

• Record configuration values such as n_estimators, max_iter, random_state, and n_jobs.

• Avoid making conclusions from a single metric.

15. Memory Monitoring: Next Planned Stage

The next stage is to add memory-related measurements using psutil. A simple first measurement can inspect the process's resident set size (RSS). RSS is the amount of physical memory currently associated with the process, although its exact interpretation depends on the operating system.

```
import os
import psutil

process = psutil.Process(os.getpid())

memory_before = (
    process.memory_info().rss / (1024 * 1024)
)

model.fit(X_train, y_train)

memory_after = (
    process.memory_info().rss / (1024 * 1024)
)

memory_increase = memory_after - memory_before
```

The conversion divides by 1024 × 1024 to express bytes as megabytes.

> **Measurement limitation:** the difference between memory_after and memory_before is not necessarily the model's exact memory requirement. It may be affected by Python's memory allocator, previously loaded libraries, operating-system behavior, caching, and memory that was allocated but not released. It is better to label the measurement clearly as an RSS change rather than claiming it is exact peak memory.

More advanced memory measurement

A stronger approach would run each model in a separate subprocess and monitor peak RSS during execution. This is more complex but can reduce contamination between models. The method should be introduced only after the basic measurement is understood.

16. Energy Measurement and Its Limitations

Energy measurement is the long-term goal of the Green AI framework, but it is more difficult than timing. A CPU-only Windows computer may not expose accurate per-process energy readings through standard Python libraries.

Possible approaches include:

• Hardware power meters for whole-system energy measurement.

• Operating-system or processor energy counters, when supported.

• External tools that expose CPU package energy or power information.

• Approximation using measured power and execution time, with clear assumptions.

Any approximation must document whether it measures CPU package energy, total system energy, or an estimated value. A laptop's battery discharge, wall power, and CPU package energy are different quantities and should not be treated as interchangeable.

17. Reproducibility and Research Methodology

A reproducible experiment should allow another person to understand what was done and repeat it under similar conditions.

Item to document	Why it matters
Dataset source and version	The data may change or be preprocessed differently.
Train-test split	Different splits can produce different accuracy values.
Random seed	Controls stochastic operations where supported.
Model parameters	Parameters influence accuracy and computational cost.
Python and package versions	Library implementations can affect results.
Hardware	CPU, RAM, and system load affect performance.
Number of runs	More runs provide better information about variability.
Metric definitions	Prevents ambiguous interpretations of memory and energy.

The project should distinguish between measured facts, interpretations, and limitations. For example, 'Random Forest took 0.189 seconds in the baseline run' is a measurement. 'Random Forest is always inefficient' is an unsupported generalization.

18. Recommended GitHub Documentation

README sections

• Project title and short description.

• Research question.

• Why Green AI matters.

• Dataset and model descriptions.

• Installation instructions.

• How to run each script.

• Metrics and their definitions.

• Result tables and plots.

• Limitations.

• Future improvements.

• Citation or references, if external research is added.

Suggested run commands

```
# Activate the virtual environment in PowerShell
.\venv\Scripts\Activate.ps1

# Verify the environment
python test_setup.py

# Run the basic timing experiment
python first_experiment.py

# Run the baseline machine-learning benchmark
python ml_baseline.py

# Create plots
python plot_results.py
```

```
# Run the repeated benchmark
python repeated_benchmark.py

# Save installed package versions
python -m pip freeze > requirements.txt
```

19. Key Lessons Learned

• A machine-learning project needs more than model training; it also needs experiment design, measurement, storage, visualization, and documentation.

• Accuracy alone is not enough for a Green AI comparison.

• Training time and prediction time are separate metrics.

• Repeated experiments are more informative than one isolated run.

• Mean and standard deviation help communicate variability.

• CPU utilization is not the same as electrical energy.

• Execution time is not the same as memory usage.

• RSS change is not automatically exact peak memory or model-only memory.

• A result should be stated in the context of its dataset, configuration, hardware, and test split.

• Good research writing reports limitations instead of hiding them.

20. Current Project Status

Area	Status
Python environment	Completed
Package installation and import test	Completed
Basic timing experiment	Completed
Dataset loading and fixed split	Completed
Three-model baseline benchmark	Completed
CSV result saving	Completed
Accuracy and training-time plots	Completed
Five-run repeated benchmark	Completed
Mean and standard deviation summary	Completed
Memory measurement	Next stage
Direct energy measurement	Future stage
Modular src/ refactor	Future improvement
Automated tests	Future improvement

21. Final Understanding

The project has moved from a simple Python timing experiment to a reproducible machine-learning benchmarking workflow. The most important conceptual shift is that a model should not be evaluated only by how accurately it predicts. A useful Green AI report also asks how much time, memory, and energy-related resource it requires, how stable the measurements are, and whether the comparison is fair.

At the current stage, the project has valid initial measurements for accuracy, training time, and prediction time. The next step is to add memory measurements carefully, followed by an energy-related method that clearly states what is measured and what is estimated.

> The strongest future version of this project will not simply announce a single winning model. Instead, it will present transparent trade-offs so that a reader can decide which model is appropriate for a particular application.

Appendix A: Quick Glossary

Term	Meaning
Feature	An input variable used by a machine-learning model.
Target/label	The value the model tries to predict.
Training set	Data used to learn model parameters.
Test set	Data reserved for evaluating generalization.
Pipeline	A sequence of preprocessing and modeling steps.
StandardScaler	Transforms features to a standardized scale.
Accuracy	Fraction of predictions that are correct.
RSS	Resident set size; process memory currently resident in physical memory.
Mean	Average of observed values.
Standard deviation	A measure of spread or variability.
Reproducibility	Ability to repeat an experiment under documented conditions.
Green AI	AI development that considers computational and environmental cost.

Appendix B: Practical Checklist

- Run the environment test without errors.
- Confirm the dataset split and random seed.
- Confirm every model uses the intended preprocessing.
- Save raw results before creating summary statistics.
- Repeat experiments instead of trusting one timing result.
- Save plots to the results/plots directory.
- Record package versions using requirements.txt.
- Label memory measurements accurately.
- Do not call time or CPU percentage direct energy measurements.
- Document limitations in the README and methodology notes.